

Compiler-Driven Parallelism in Modern Fortran using DO CONCURRENT

Technical Report

Author: Shahid Maqbool
Version: 1.0.0
Date: July 17, 2026
GitHub Repository: [:/https://github.com/Shahid718/Compiler-Driven-Parallelism-in-Modern-Fortran-using-DO-CONCURRENT](https://github.com/Shahid718/Compiler-Driven-Parallelism-in-Modern-Fortran-using-DO-CONCURRENT)
License: MIT License

Abstract

This document describes the compiler-driven parallelism using the Fortran 2008 DO CONCURRENT construct in a modern Fortran Cahn-Hilliard solver. The report summarizes compiler configuration, benchmarking methodology, and performance analysis on Intel oneAPI and NVIDIA HPC SDK compilers. The primary objective is to demonstrate portable, performance-portable parallelism through standard language constructs, avoiding vendor-specific extensions or external libraries.

1 Introduction

1.1 Objectives

The objective of this repository is to investigate **portable compiler-driven parallelism** in Modern Fortran using only standard language constructs. Specifically, this work focuses on the `DO CONCURRENT` construct introduced in Fortran 2008 and significantly enhanced in Fortran 2018.

This repository presents a case study of a CahnHilliard phase-field solver, designed to explore how compilers can leverage `DO CONCURRENT` to generate parallel code across different hardware architectures without modifying the source code.

1.2 Motivation

The HPC landscape is increasingly heterogeneous, with CPUs, GPUs, and accelerators coexisting in modern supercomputing environments. Achieving performance portability where a single codebase performs well across diverse architectures remains a significant challenge. Traditional approaches often rely on vendor-specific extensions (e.g., OpenMP, CUDA, OpenACC) or require code duplication for different backends.

`DO CONCURRENT` offers an alternative: a standard language construct that expresses loop-level parallelism, allowing the compiler to decide to parallelize the code. This approach promises:

- **Why `DO CONCURRENT`:** A Fortran-standard way to express data-parallel loops without relying on compiler directives or external libraries. `DO CONCURRENT` automatically enables the compiler to analyze and parallelize loops.
- **Why compiler parallelization:** Delegating parallelism decisions to the compiler reduces development overhead and ensures that optimizations remain consistent across platforms as compiler technologies evolve.
- **Why portability:** A single source code, compiled with different compilers, can target different architectures from multi-core CPUs to GPUs without modifications.
- **Why performance portability:** By abstracting the parallelization strategy, `DO CONCURRENT` allows the compiler to apply architecture-specific optimizations (e.g., vectorization, auto-parallelization, GPU offloading) transparently.

1.3 Contributions

This work makes the following contributions:

- **Modern Fortran 2008 implementation:** A clean, modular, and maintainable implementation of the CahnHilliard equation using object-oriented Fortran features.
- **Portable parallelism:** Leveraging `DO CONCURRENT` for portable parallelism across CPU and GPU architectures without compiler directives.
- **CMake build system:** A robust build system that simplifies compilation across different platforms and compilers.
- **Single and double precision support:** Configurable precision to demonstrate numerical accuracy and performance trade-offs.
- **Intel ifx benchmarks:** Comprehensive performance measurements using the Intel oneAPI Fortran compiler.

- **NVIDIA nvfortran benchmarks:** Performance analysis using the NVIDIA HPC SDK Fortran compiler.
- **Performance analysis:** Detailed analysis of strong scaling, and compiler-specific optimizations.

1.4 Report Structure

The remainder of this report is organized as follows: Section 2 presents the mathematical model of the CahnHilliard equation. Section 4 describes the software architecture and implementation details. Section 8 outlines the benchmarking methodology and compiler configurations. Section 9 presents the performance results and analysis. Finally, Section 11 concludes the report and discusses future work.

2 Mathematical Model

The CahnHilliard equation models phase separation and is a fourth-order nonlinear partial differential equation (PDE). In its dimensionless form, it is given by:

$$\frac{\partial \phi}{\partial t} = M \nabla^2 \left(\frac{\partial f}{\partial \phi} - \kappa^2 \nabla^2 \phi \right) \quad (1)$$

where:

- ϕ is the conserved order parameter (concentration field),
- M is the mobility,
- $f(\phi)$ is the homogeneous free energy density,
- κ is the gradient energy coefficient,
- ∇^2 is the Laplacian operator.

3 Numerical Implementation

The CahnHilliard equation is discretized using finite differences on a uniform grid. An explicit Euler time-stepping scheme is used for temporal discretization, with central differences for spatial derivatives.

For a 2D domain with grid spacing Δx and Δy , the Laplacian operator is approximated as:

$$\nabla^2 \phi_{i,j} \approx \frac{\phi_{i+1,j} - 2\phi_{i,j} + \phi_{i-1,j}}{\Delta x^2} + \frac{\phi_{i,j+1} - 2\phi_{i,j} + \phi_{i,j-1}}{\Delta y^2} \quad (2)$$

The fourth-order term $\nabla^4 \phi$ is computed by applying the Laplacian twice:

$$\nabla^4 \phi_{i,j} = \nabla^2 (\nabla^2 \phi_{i,j}) \quad (3)$$

Using explicit Euler time-stepping:

$$\phi_{i,j}^{n+1} = \phi_{i,j}^n + \Delta t M \nabla^2 \left(\frac{\partial f}{\partial \phi} - \kappa^2 \nabla^2 \phi_{i,j}^n \right) \quad (4)$$

where Δt is the time step size.

Periodic boundary conditions are applied on all boundaries, commonly used in phase-field simulations to mimic bulk behavior. For a domain of size $N_x \times N_y$, the boundary conditions are:

$$\phi_{0,j} = \phi_{N_x,j} \quad \forall j \in [0, N_y - 1] \quad (5)$$

$$\phi_{N_x+1,j} = \phi_{1,j} \quad \forall j \in [0, N_y - 1] \quad (6)$$

$$\phi_{i,0} = \phi_{i,N_y} \quad \forall i \in [0, N_x - 1] \quad (7)$$

$$\phi_{i,N_y+1} = \phi_{i,1} \quad \forall i \in [0, N_x - 1] \quad (8)$$

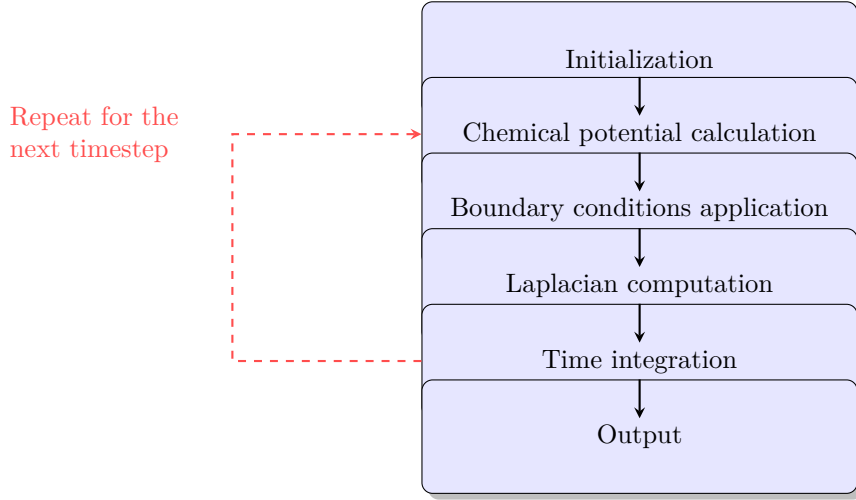


Figure 1: Numerical implementation flowchart for the Cahn-Hilliard solver.

4 Software Architecture

The solver adopts a modular design in which computational kernels are separated from supporting infrastructure. The computational kernel modules implement the numerical operations and contain all DO CONCURRENT parallel regions, while the infrastructure modules manage precision, grid initialization, timing, I/O, error handling, and performance reporting etc. This organization improves readability, maintainability, and facilitates future optimization of the performance-critical kernels. Figure 2 illustrates the overall software architecture.

5 Compiler Configurations

Two compiler suites are evaluated:

Table 1: Release compiler configurations used for benchmarking.

Category	Intel oneAPI ifx	NVIDIA HPC SDK nvfortran
Optimization	/O2, /QxHost	-O3
Parallelization	/Qopenmp	-stdpar=multicore
Optimization Reports	/Qopt-report:3	-Minfo=all

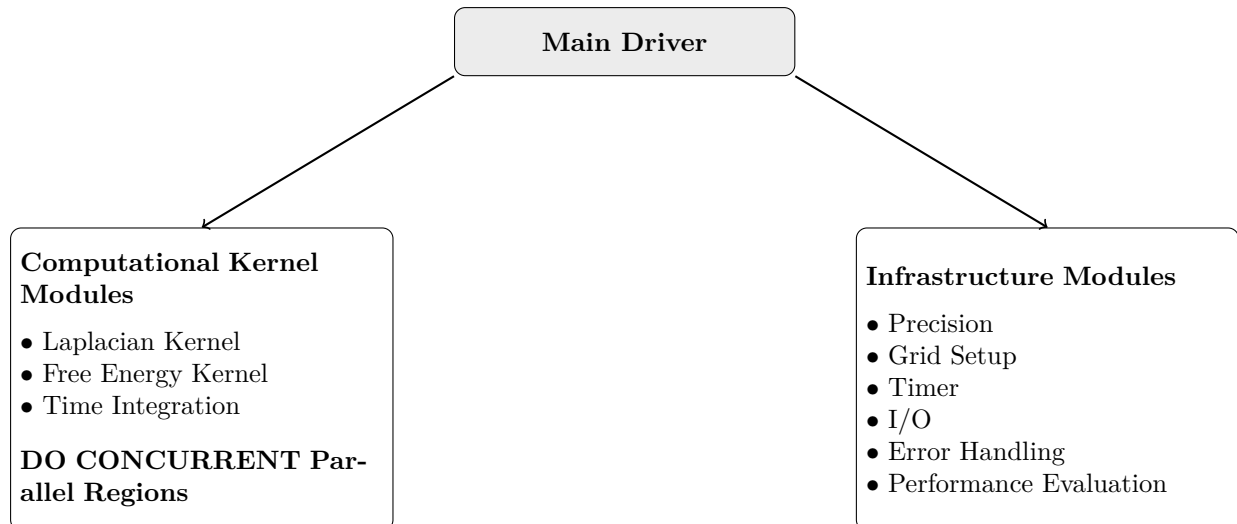


Figure 2: Software architecture of the phase-field solver. The computational kernels contain the `DO CONCURRENT` parallel regions, while the infrastructure modules provide precision control, grid management, timing, input/output, error handling, and performance reporting etc.

Note: The repository also provides dedicated Debug, RelWithDebInfo, and Profile build configurations. All performance results presented in this report were obtained using the Release configuration.

6 Parallel Programming Strategy

This section describes the parallel programming strategy employed in this work, contrasting traditional approaches with the modern Fortran `DO CONCURRENT` construct.

6.1 Traditional Approach: OpenMP Directives

Traditionally, loop-level parallelism in Fortran has been achieved using OpenMP directives. For example, a parallel loop is expressed as:

```

1      !$OMP PARALLEL DO
2      do i = 1, n
3      a(i) = b(i) + c(i)
4      end do
5      !$OMP END PARALLEL DO
  
```

Listing 1: Traditional OpenMP parallel loop in Fortran.

While effective, this approach has several limitations:

- Requires compiler directives that are not part of the Fortran standard.
- Ties the code to a specific external parallelization model.
- leads to code clutter with numerous directives.

6.2 Modern Approach: `DO CONCURRENT`

The Fortran 2008 standard introduced `DO CONCURRENT`, a native language construct for expressing loop-level parallelism:

```

1      do concurrent (i = 1:n)
2      a(i) = b(i) + c(i)
3      end do

```

Listing 2: Modern Fortran DO CONCURRENT loop.

6.3 Parallelization Flow

The DO CONCURRENT approach enables a clean separation of concerns between the programmer and the compiler:

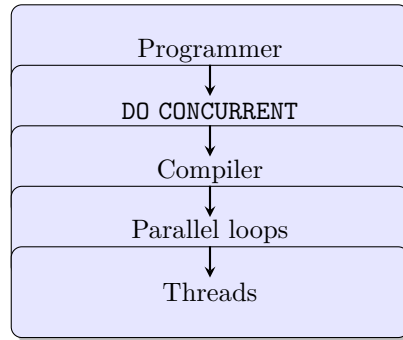


Figure 3: Parallelization flow with DO CONCURRENT.

The workflow is as follows:

1. The **programmer** expresses the loop using the standard DO CONCURRENT construct.
2. The **compiler** analyzes the loop and determines the optimal parallelization strategy.
3. The compiler generates **parallel loops**.
4. The parallel loops are executed across available **threads** or processing units.

7 CMake Build System

The repository uses CMake as the build system, providing:

- Cross-platform support (Linux, Windows).
- Support for multiple compilers (Intel ifx, NVIDIA nvfortran).
- Automatic detection of compiler capabilities.
- Configuration of single/double precision at compile time.

8 Benchmarking Methodology

All benchmarks were performed using identical source code, numerical parameters, and compiler-driven parallelization via the standard Fortran DO CONCURRENT construct. Only the compiler, operating system, hardware platform, thread count, and floating-point precision were varied during the performance evaluation.

Table 2: Benchmark platforms used for performance evaluation.

Component	Platform 1	Platform 2
Processor	Intel Core i5 (8th Generation)	AMD Ryzen 9 3900XT
Operating System	Windows 11	Ubuntu (WSL2)
Compiler	Intel oneAPI ifx 2026	NVIDIA HPC SDK nvfortran 24.5
Physical Cores	4	12
Logical Threads	8	24 (SMT Enabled)
Parallel Model	DO CONCURRENT	DO CONCURRENT
Precision Modes	Single, Double	Single, Double
Grid Sizes	1000 ² –4000 ²	1000 ² –4000 ²
Time steps	2,000	2,000
Benchmark Metrics	Execution Time, MLUPS	Execution Time, MLUPS

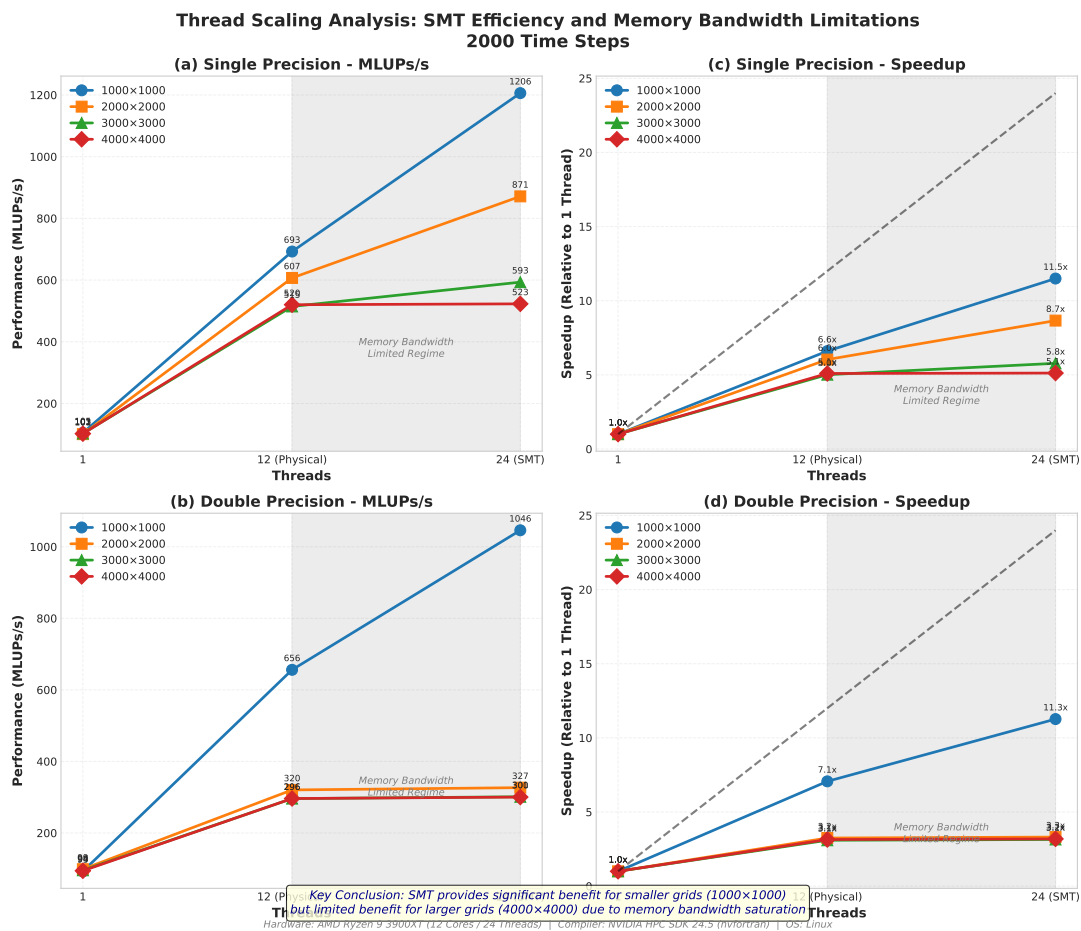
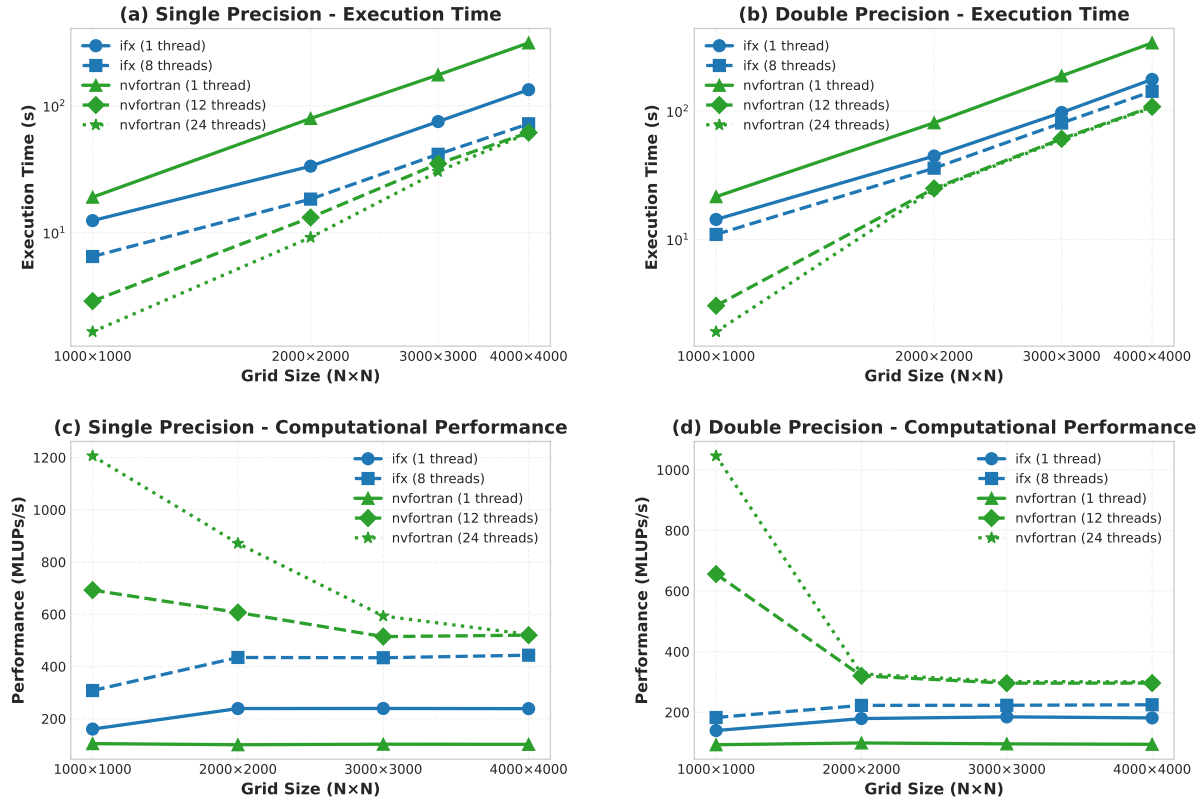


Figure 4: Strong scaling analysis of the compiler-driven DO CONCURRENT implementation using the NVIDIA HPC SDK (nvfortran) on an AMD Ryzen 9 3900XT processor. The figure compares performance across 1, 12, and 24 threads, highlighting the influence of SMT and memory-bandwidth limitations for a fixed workload of 2000 time steps.

9 Performance Results

Figure 4 shows the strong scaling results for the NVIDIA HPC SDK (nvfortran) on an AMD Ryzen 9 3900XT processor.

Cahn-Hilliard Benchmark: Performance Comparison



Intel ifx: Windows 11, 8 threads | NVIDIA nvfortran: Linux, 12/24 threads

Figure 5: Execution time and computational performance comparison between Intel ifx and NVIDIA nvfortran for single- and double-precision implementations.

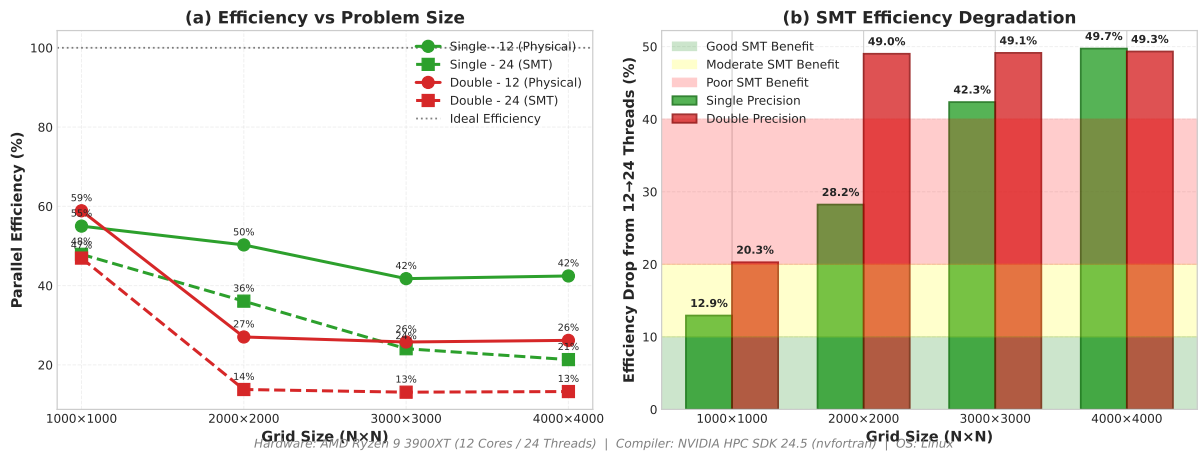
SMT Efficiency Analysis: Memory Bandwidth Limitations
2000 Time Steps

Figure 6: SMT scalability analysis showing the impact of memory-bandwidth limitations on the compiler-driven DO CONCURRENT implementation using NVIDIA nvfortran on an AMD Ryzen 9 3900XT platform.

10 Discussion

Figure 4 shows the strong scaling performance of the DO CONCURRENT solver over 2000 time steps. The execution time decreases substantially from 1 to 12 threads, demonstrating effective compiler-driven parallelization. However, beyond 12 physical cores, the additional SMT threads provide only marginal improvements due to memory bandwidth limitations. This behavior is typical for memory-bound applications and indicates that the solver’s performance is constrained by data movement rather than compute capacity.

Single precision (Figure 5) consistently delivers higher throughput than double precision for both compilers. The reduction in performance observed for double precision is expected because each numerical update requires twice the memory traffic. As the grid size increases, the working set exceeds the processor caches, making memory bandwidth the dominant performance constraint. Consequently, the performance gap between single and double precision becomes more apparent for larger computational domains. An important observation is that the achieved MLUPS gradually decreases as the grid size increases for all parallel configurations. Although larger problems provide more computational work, they also place greater pressure on the memory hierarchy, reducing arithmetic intensity and increasing memory-access latency. This behavior is characteristic of memory-bound stencil computations, where overall performance is increasingly limited by memory bandwidth rather than raw computational capability.

Figure 5 also shows Intel ifx trailing NVIDIA nvfortran in absolute throughput across both precisions. This gap should not be read as a statement about DO CONCURRENT parallelization itself; optimization reports generated during compilation (`-qopt-report=3`) confirm that ifx genuinely transforms DO CONCURRENT loops into multithreaded regions, in the same way nvfortran does. Rather, the two platforms in Table 2 differ in more than compiler: Platform 1 has 4 physical cores against Platform 2’s 12, and runs natively on Windows 11 rather than native Linux. With three times fewer physical cores available, and with DO CONCURRENT regions launched fresh each time step over 2,000 time steps, Platform 1 has both a lower ceiling on achievable parallelism and proportionally more thread fork/join overhead to amortize per unit of useful work. The comparison in Figure 5 is therefore illustrative of two representative platforms rather than a controlled, single-variable measurement of compiler performance; isolating the compiler as the sole independent variable, by benchmarking ifx on the same AMD Ryzen 9 3900XT platform used for the nvfortran results, is identified as near-term future work.

For Figure 6 with the smallest problem size (1000×1000), enabling SMT provides a noticeable improvement in computational efficiency. This indicates that the processor is able to utilize otherwise idle execution resources when the workload is relatively small, allowing the additional logical threads to contribute useful work. As the problem size increases, however, the benefit of SMT decreases substantially. For grid sizes of 2000×2000 and larger, the improvement obtained by increasing the thread count from 12 to 24 becomes progressively smaller, and the efficiency gap between physical-core execution and SMT execution widens. This trend is observed consistently for both single- and double-precision simulations.

11 Conclusion

This report has presented the design and implementation of a modern Fortran CahnHilliard solver using the Fortran 2008 DO CONCURRENT construct. The primary objective—demonstrating portable, compiler-driven parallelism without vendor-specific directives—was achieved: a single, unmodified source tree was compiled and parallelized by two independent compiler vendors (Intel oneAPI and NVIDIA HPC SDK) using only standard Fortran.

key findings:

- `DO CONCURRENT` provides a portable, standard-compliant path to loop-level parallelism in Fortran, eliminating the need for vendor-specific directives such as OpenMP or OpenACC. This was confirmed directly, not just inferred from timing: compiler optimization reports (`-qopt-report`) show `DO CONCURRENT` loops being outlined into genuine multithreaded regions on both compilers.
- Both compilers deliver substantial multi-core speedup from physical cores alone up to $7.1 \times$ on 12 physical cores (NVIDIA HPC SDK, double precision, Figure 4 confirming that `DO CONCURRENT` is a practical, working parallelization strategy for this class of stencil computation.
- Scaling is memory-bandwidth-limited beyond the physical core count, not compute-limited. Parallel efficiency on the NVIDIA HPC SDK platform ranges from 59 % down to 13 to 26 % at larger grid sizes with SMT enabled (Figure 6), and additional logical (SMT) threads provide only marginal benefit once the working set exceeds cache capacity. This is the expected, textbook signature of a memory-bound stencil code, not a shortcoming of `DO CONCURRENT` itself it reflects the arithmetic intensity of the CahnHilliard update, not the parallelization mechanism.
- The single-precision-vs-double-precision performance gap widens with grid size (Figure 6), consistent with double precision’s doubled memory traffic becoming the dominant cost as problems grow beyond cache.
- The cross-compiler comparison in Figure 6 was run on two different hardware platforms (Table 2) and should be read as an illustrative comparison rather than a controlled one; isolating the compiler as the sole independent variable is identified as near-term future work below.

11.1 Future work

- Repeat the Intel oneAPI benchmark on the same AMD Ryzen 9 3900XT platform used for the NVIDIA HPC SDK results, isolating compiler choice as the only variable and enabling a directly controlled ifx-vs-nvfortran comparison.
- Optimize the periodic-boundary indexing in the Laplacian kernel (currently branch-computed per element) using ghost/halo cells, removing the gather-load penalty identified in the Intel vectorization reports and re-evaluating its contribution to overall performance.
- Extend the test suite to exercise the production ‘src/’ modules directly, including a mass-conservation check (invariance over time) as a numerical correctness safeguard.
- Extend the solver to 3D grids.
- Explore hybrid MPI + `DO CONCURRENT` parallelization for distributed-memory systems.
- Pursue native GPU offloading via `DO CONCURRENT`, building on the CPU-parallel results presented here.